

```
#include <BLEDevice.h>
```

```
#include <BLEUtils.h>
```

```
#include <BLEServer.h>
```

```
#include <BLE2902.h>
```

```
#include <ArduinoJson.h>
```

```
#include <Preferences.h>
```

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
// ----- CONFIG -----
```

```
#define SERVICE_UUID    "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
```

```
#define CMD_CHAR_UUID    "beb5483e-36e1-4688-b7f5-ea07361b26a8"
```

```
#define STAT_CHAR_UUID    "d06b9f92-2b3d-4f6c-9b3f-2e7a1a5b4f9c"
```

```
const char* BLE_NAME = "SmartMed-ESP32";
```

```
// Pins (adjust if required)
```

```
const int MOTOR_IN1 = 12;
```

```
const int MOTOR_IN2 = 14;
```

```
const int IR_DISP_PIN = 34;
```

```
const int IR_HAND_PIN = 35;
```

```
const int BUZZER_PIN = 25;
```

```
const int LED_MISS_PIN = 27;
```

```
// LCD (change address if necessary)
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```

// Motor burst configuration

const int BURST_TIME_MS = 50;

const int PAUSE_TIME_MS = 150;

const int MAX_BURSTS = 20;


// Hand detection timing

const unsigned long HAND_CYCLE_MS = 30000UL; // 30s

const int HAND_CYCLES = 2; // total 60s

const unsigned long SCHEDULER_CHECK_MS = 1500UL; // scheduler tick (ms)


// Preferences (NVS)

Preferences prefs;

const char* PREF_KEY_REM = "reminders"; // stores JSON array of reminders

const char* PREF_KEY_HIST = "history"; // stores JSON array history

const char* PREF_KEY_SEQ = "seqid"; // next id integer


// BLE globals

BLECharacteristic *pCmdChar = nullptr;

BLECharacteristic *pStatChar = nullptr;

BLEServer *pServer = nullptr;

bool deviceConnected = false;


// Runtime

unsigned long lastSchedulerRun = 0;


// ----- Helpers: NVS JSON storage -----

String loadPrefsString(const char* key) {

    if (!prefs.isKey(key)) return "[]";

```

```

String s = prefs.getString(key, "");
if (s.length() == 0) return "";
return s;
}

```

```

void savePrefsString(const char* key, const String &s) {
    prefs.putString(key, s.c_str());
}

```

```

// generate incremental id
String nextId() {
    long seq = prefs.getLong(PREF_KEY_SEQ, 1);
    String id = String(seq);
    prefs.putLong(PREF_KEY_SEQ, seq + 1);
    return id;
}

```

```

// ----- BLE helpers -----
void notifyJson(const JsonVariant &v) {
    String out;
    serializeJson(v, out);
    if (pStatChar && deviceConnected) {
        // setValue accepts (const uint8_t*, size) or std::string — use bytes variant
        pStatChar->setValue((uint8_t*)out.c_str(), out.length());
        pStatChar->notify();
    }
    Serial.println("NOTIFY -> " + out);
}

```

```

void notifyTypeDetail(const char* type, const char* detail=nullptr) {
    StaticJsonDocument<256> doc;
    doc["type"] = type;
    if (detail) doc["detail"] = detail;
    doc["time_ms"] = millis();
    notifyJson(doc.as<JsonVariant>());
}

```

```

// ----- Motor & sensor functions -----

```

```

void motorStop() {
    digitalWrite(MOTOR_IN1, LOW);
    digitalWrite(MOTOR_IN2, LOW);
}

```

```

void motorBurstOnce() {
    digitalWrite(MOTOR_IN1, HIGH);
    digitalWrite(MOTOR_IN2, LOW);
    delay(BURST_TIME_MS);
    motorStop();
}

```

```

bool attemptDispenseBurst() {
    bool detected = false;
    for (int i=0;i<MAX_BURSTS;i++) {
        motorBurstOnce();
        // check pill sensor during small window
        unsigned long t0 = millis();

```

```

while (millis() - t0 < 30) {
    if (digitalRead(IR_DISP_PIN) == LOW) { detected = true; break; }
    delay(5);
}
if (detected) break;
// pause
unsigned long t1 = millis();
while (millis() - t1 < PAUSE_TIME_MS) {
    if (digitalRead(IR_DISP_PIN) == LOW) { detected = true; break; }
    delay(10);
}
if (detected) break;
}
motorStop();
return detected;
}

// Wait for hand detection up to HAND_CYCLES * HAND_CYCLE_MS; beep the buzzer
periodically.

// Return true if detected.
bool waitForHandDetection() {
    for (int cycle=0; cycle<HAND_CYCLES; cycle++) {
        unsigned long start = millis();
        while (millis() - start < HAND_CYCLE_MS) {
            // beep pattern 300ms on, 700ms off
            digitalWrite(BUZZER_PIN, HIGH);
            delay(300);
            digitalWrite(BUZZER_PIN, LOW);

```

```

    // brief check after beep
    if (digitalRead(IR_HAND_PIN) == LOW) return true;
    delay(700);
}
}
return false;
}

```

```

// ----- Reminders management -----
String remindersToString() {
    return loadPrefsString(PREF_KEY_REM);
}

```

```

void saveRemindersFromDoc(JsonArray &arr) {
    String out;
    serializeJson(arr, out);
    savePrefsString(PREF_KEY_REM, out);
}

```

```

void saveHistoryFromDoc(JsonArray &arr) {
    String out;
    serializeJson(arr, out);
    savePrefsString(PREF_KEY_HIST, out);
}

```

```

JsonArray loadRemindersDoc(JsonDocument &doc) {
    String s = remindersToString();
    DeserializationError err = deserializeJson(doc, s);
}

```

```

if (err) {
    doc.clear();
    return doc.to<JsonArray>();
}
return doc.as<JsonArray>();
}

```

```

JsonArray loadHistoryDoc(JsonDocument &doc) {
    String s = loadPrefsString(PREF_KEY_HIST);
    DeserializationError err = deserializeJson(doc, s);
    if (err) {
        doc.clear();
        return doc.to<JsonArray>();
    }
    return doc.as<JsonArray>();
}

```

// add history entry

```

void pushHistory(const char* id, const char* patient, const char* med, unsigned long ts,
const char* result) {

    StaticJsonDocument<1024> histDoc;

    JsonArray histArr = loadHistoryDoc(histDoc);
    JsonObject h = histArr.createNestedObject();

    h["id"] = id;
    h["patient"] = patient;
    h["med"] = med;
    h["timestampMs"] = ts;
    h["result"] = result;
}

```

```

h["whenMs"] = millis();

saveHistoryFromDoc(histArr);
}

// ----- BLE callbacks & command processing -----

class ServerCallbacks : public BLEServerCallbacks {

    void onConnect(BLEServer* pServer) {

        deviceConnected = true;

        notifyTypeDetail("ble_connected", nullptr);

        Serial.println("BLE connected");
    }

    void onDisconnect(BLEServer* pServer) {

        deviceConnected = false;

        notifyTypeDetail("ble_disconnected", nullptr);

        Serial.println("BLE disconnected");
    }

};

class CmdCallbacks : public BLECharacteristicCallbacks {

    void onWrite(BLECharacteristic* pChar) {

        // FIXED — no std::string anywhere

        String s = String(pChar->getValue().c_str());

        if (s.length() == 0) return;

        Serial.println("CMD <- " + s);
    }
};

```



```

StaticJsonDocument<1536> doc;

DeserializationError err = deserializeJson(doc, s);

if (err) {
    notifyTypeDetail("error", "bad_json");
    Serial.println("Bad JSON");
    return;
}

const char* cmd = doc["cmd"] | "";
// ----- schedule -----
if (strcmp(cmd, "schedule") == 0) {
    if (!doc.containsKey("timestamp") || !doc.containsKey("patient") ||
!doc.containsKey("med")) {
        notifyTypeDetail("error", "missing_fields");
        return;
    }

    unsigned long timestampMs = (unsigned long) doc["timestamp"].as<unsigned long
long>();

    // optional: client current unix ms
    unsigned long nowClientMs = 0;
    bool haveNow = false;
    if (doc.containsKey("now")) {
        nowClientMs = (unsigned long) doc["now"].as<unsigned long long>();
        haveNow = true;
    }

    const char* patient = doc["patient"] | "";
    const char* med = doc["med"] | "";
    const char* dosage = doc["dosage"] | "";

```

```

// append to reminders and compute dueAtLocalMs if 'now' provided
StaticJsonDocument<2048> rdoc;

JsonArray arr = loadRemindersDoc(rdoc);

JsonObject item = arr.createNestedObject();

String id = nextId();

item["id"] = id.c_str();

item["patient"] = patient;

item["med"] = med;

item["dosage"] = dosage;

item["timestampMs"] = (unsigned long long) timestampMs;

item["status"] = "pending";

item["createdAt"] = millis();

if (haveNow) {
    // compute dueAtLocalMs = millis() + (timestamp - nowClientMs)
    long long diff = (long long)timestampMs - (long long)nowClientMs;
    if (diff < 0) diff = 0;

    unsigned long dueLocal = millis() + (unsigned long)diff;

    item["dueAtLocalMs"] = (unsigned long long)dueLocal;

    item["nowSentMs"] = (unsigned long long)nowClientMs;
}

saveRemindersFromDoc(arr);


// confirm and send list
StaticJsonDocument<512> out;

out["type"] = "scheduled";

out["id"] = id.c_str();

out["patient"] = patient;

```

```
out["med"] = med;
notifyJson(out.as<JsonVariant>());
```

```
StaticJsonDocument<3072> listDoc;
JsonArray listArr = loadRemindersDoc(listDoc);
StaticJsonDocument<3072> wrap;
wrap["type"] = "list";
wrap["count"] = listArr.size();
wrap["reminders"] = listArr;
notifyJson(wrap.as<JsonVariant>());
return;
}
```

```
// ----- list -----
if (strcmp(cmd, "list") == 0) {
    StaticJsonDocument<3072> listDoc;
    JsonArray listArr = loadRemindersDoc(listDoc);
    StaticJsonDocument<3072> wrap;
    wrap["type"] = "list";
    wrap["count"] = listArr.size();
    wrap["reminders"] = listArr;
    notifyJson(wrap.as<JsonVariant>());
    return;
}
```

```
// ----- delete -----
if (strcmp(cmd, "delete") == 0) {
    if (!doc.containsKey("id")) { notifyTypeDetail("error", "id_required"); return; }
```

```

const char* id = doc["id"] | "";
StaticJsonDocument<4096> remDoc;
JsonArray arr = loadRemindersDoc(remDoc);
StaticJsonDocument<4096> newDoc;
JsonArray newArr = newDoc.to<JsonArray>();
for (JsonObject r : arr) {
    if (strcmp(r["id"] | "", id) != 0) {
        JsonObject n = newArr.createNestedObject();
        n["id"] = r["id"];
        n["patient"] = r["patient"];
        n["med"] = r["med"];
        n["dosage"] = r["dosage"];
        n["timestampMs"] = r["timestampMs"];
        n["status"] = r["status"];
        n["createdAt"] = r["createdAt"];
        if (r.containsKey("dueAtLocalMs")) n["dueAtLocalMs"] = r["dueAtLocalMs"];
    }
}
saveRemindersFromDoc(newArr);
notifyTypeDetail("deleted", id);
// send updated list
StaticJsonDocument<3072> wrap;
JsonArray listArr = loadRemindersDoc(wrap);
StaticJsonDocument<3072> out;
out["type"] = "list";
out["count"] = listArr.size();
out["reminders"] = listArr;
notifyJson(out.as<JsonVariant>());

```

```

    return;
}

// ----- startNow -----
if (strcmp(cmd, "startNow") == 0) {
    if (doc.containsKey("id")) {
        const char* id = doc["id"] | "";

        StaticJsonDocument<4096> remDoc;

        JsonArray arr = loadRemindersDoc(remDoc);

        bool found = false;

        for (JsonObject r : arr) {
            if (strcmp(r["id"] | "", id) == 0) {
                r["status"] = "running";

                r["dueAtLocalMs"] = (unsigned long long)(millis() + 100);

                found = true;

                break;
            }
        }

        saveRemindersFromDoc(arr);

        if (found) notifyTypeDetail("starting", id);

        else notifyTypeDetail("error", "not_found");

        return;
    } else {

        // manual immediate schedule

        StaticJsonDocument<2048> doc2;

        JsonArray arr = loadRemindersDoc(doc2);

        JsonObject item = arr.createNestedObject();

        String idn = nextId();

```

```

    item["id"] = idn.c_str();
    item["patient"] = doc["patient"] | "";
    item["med"] = doc["med"] | "";
    item["dosage"] = doc["dosage"] | "";
    item["dueAtLocalMs"] = (unsigned long long)(millis() + 100);
    item["status"] = "pending";
    item["createdAt"] = millis();
    saveRemindersFromDoc(arr);
    notifyTypeDetail("manual_scheduled", nullptr);
    // send updated list
    StaticJsonDocument<3072> wrap;
    JsonArray listArr = loadRemindersDoc(wrap);
    StaticJsonDocument<3072> out;
    out["type"] = "list";
    out["count"] = listArr.size();
    out["reminders"] = listArr;
    notifyJson(out.as<JsonVariant>());
    return;
}
}

```

```

// ----- cancel -----
if (strcmp(cmd, "cancel") == 0) {
    if (doc.containsKey("id")) {
        const char* id = doc["id"] | "";
        StaticJsonDocument<4096> remDoc;
        JsonArray arr = loadRemindersDoc(remDoc);
        for (JsonObject r : arr) {

```

```

        if (strcmp(r["id"] | "", id) == 0) {
            r["status"] = "canceled";
            break;
        }
    }

    saveRemindersFromDoc(arr);
    notifyTypeDetail("canceled", id);
} else {
    StaticJsonDocument<4096> remDoc;
    JsonArray arr = loadRemindersDoc(remDoc);
    for (JsonObject r : arr) {
        if (strcmp(r["status"] | "pending", "pending") == 0) r["status"] = "canceled";
    }
    saveRemindersFromDoc(arr);
    notifyTypeDetail("canceled_all", nullptr);
}

// send updated list
StaticJsonDocument<3072> wrap;
JsonArray listArr = loadRemindersDoc(wrap);
StaticJsonDocument<3072> out;
out["type"] = "list";
out["count"] = listArr.size();
out["reminders"] = listArr;
notifyJson(out.as<JsonVariant>());
return;
}

// ----- getHistory -----

```

```

if (strcmp(cmd, "getHistory") == 0) {
    StaticJsonDocument<4096> histDoc;
    JsonArray h = loadHistoryDoc(histDoc);
    StaticJsonDocument<4096> out;
    out["type"] = "history";
    out["count"] = h.size();
    out["history"] = h;
    notifyJson(out.as<JsonVariant>());
    return;
}

// ----- clearHistory -----
if (strcmp(cmd, "clearHistory") == 0) {
    StaticJsonDocument<4> empty;
    JsonArray ar = empty.to<JsonArray>();
    saveHistoryFromDoc(ar);
    notifyTypeDetail("history_cleared", nullptr);
    return;
}

// unknown
notifyTypeDetail("error", "unknown_cmd");
}

};

// ----- Scheduler: check pending reminders and trigger them -----
void runSchedulerTick() {
    if (millis() - lastSchedulerRun < SCHEDULER_CHECK_MS) return;

```



```
lastSchedulerRun = millis();
```

```
StaticJsonDocument<4096> doc;
```

```
JsonArray arr = loadRemindersDoc(doc);
```

```
if (arr.size() == 0) return;
```

```
for (JsonObject r : arr) {
```

```
    const char* status = r["status"] | "pending";
```

```
    if (strcmp(status, "pending") != 0) continue;
```

```
    unsigned long dueAtLocal = 0;
```

```
    if (r.containsKey("dueAtLocalMs")) {
```

```
        dueAtLocal = (unsigned long) r["dueAtLocalMs"].as<unsigned long long>();
```

```
    } else {
```

```
        // no local due time computed => skip (frontend must send 'now' when scheduling)
```

```
        continue;
```

```
    }
```

```
if (millis() >= dueAtLocal) {
```

```
    // mark running
```

```
    r["status"] = "running";
```

```
    saveRemindersFromDoc(arr);
```

```
    // notify start
```

```
    StaticJsonDocument<512> ev;
```

```
    ev["type"] = "reminder_started";
```

```
    ev["id"] = r["id"];
```

```
    ev["patient"] = r["patient"];
```

```

ev["med"] = r["med"];
notifyJson(ev.as<JsonVariant>());

// perform dispense
lcd.clear();
lcd.print("Dispensing...");
lcd.setCursor(0,1);
String med = String((const char*)r["med"]);
lcd.print(med.substring(0,16));

notifyTypeDetail("dispense_attempt", nullptr);
bool dropped = attemptDispenseBurst();
if (!dropped) {
    r["status"] = "done";
    saveRemindersFromDoc(arr);
    pushHistory(r["id"] | "", r["patient"] | "", r["med"] | "", (unsigned
long)r["timestampMs"].as<unsigned long long>(), "missed");
    notifyTypeDetail("dispense_failed", "no_pill");
    notifyTypeDetail("missed", nullptr);
    digitalWrite(LED_MISS_PIN, HIGH);
    lcd.clear(); lcd.print("DROP FAIL");
    delay(1500);
    continue;
}

// pill dropped -> check hand
lcd.clear(); lcd.print("Check hand...");
bool taken = waitForHandDetection();

```

```

    if (taken) {
        r["status"] = "done";
        saveRemindersFromDoc(arr);
        pushHistory(r["id"] | "", r["patient"] | "", r["med"] | "", (unsigned
long)r["timestampMs"].as<unsigned long long>(), "taken");
        notifyTypeDetail("taken", nullptr);
        digitalWrite(LED_MISS_PIN, LOW);
        lcd.clear(); lcd.print("Taken");
    } else {
        r["status"] = "done";
        saveRemindersFromDoc(arr);
        pushHistory(r["id"] | "", r["patient"] | "", r["med"] | "", (unsigned
long)r["timestampMs"].as<unsigned long long>(), "missed");
        notifyTypeDetail("missed", nullptr);
        digitalWrite(LED_MISS_PIN, HIGH);
        lcd.clear(); lcd.print("MISSED");
    }
    delay(1200);
    lcd.clear();
    lcd.print("Waiting...");
}
}
}

```

```

// ----- Setup & loop -----

```

```

void setup() {
    Serial.begin(115200);
    delay(100);
}

```

```
// pins  
pinMode(MOTOR_IN1, OUTPUT);  
pinMode(MOTOR_IN2, OUTPUT);  
pinMode(IR_DISP_PIN, INPUT);  
pinMode(IR_HAND_PIN, INPUT);  
pinMode(BUZZER_PIN, OUTPUT);  
pinMode(LED_MISS_PIN, OUTPUT);
```

```
digitalWrite(MOTOR_IN1, LOW);  
digitalWrite(MOTOR_IN2, LOW);  
digitalWrite(BUZZER_PIN, LOW);  
digitalWrite(LED_MISS_PIN, LOW);
```

```
// LCD  
lcd.init();  
lcd.backlight();  
lcd.clear();  
lcd.print("SmartMed BLE");  
lcd.setCursor(0,1);  
lcd.print("Ready...");  
delay(800);  
lcd.clear();
```

```
// preferences start  
prefs.begin("smartmed", false);  
if (!prefs.isKey(PREF_KEY_SEQ)) prefs.putLong(PREF_KEY_SEQ, 1);  
if (!prefs.isKey(PREF_KEY_REM)) savePrefsString(PREF_KEY_REM, "");  
if (!prefs.isKey(PREF_KEY_HIST)) savePrefsString(PREF_KEY_HIST, "");
```

```

// BLE init

BLEDevice::init(BLE_NAME);

pServer = BLEDevice::createServer();

pServer->setCallbacks(new ServerCallbacks());


BLEService *pService = pServer->createService(SERVICE_UUID);


pCmdChar = pService->createCharacteristic(CMD_CHAR_UUID,
BLECharacteristic::PROPERTY_WRITE);

pCmdChar->setCallbacks(new CmdCallbacks());


pStatChar = pService->createCharacteristic(STAT_CHAR_UUID,
BLECharacteristic::PROPERTY_NOTIFY | BLECharacteristic::PROPERTY_READ);

pStatChar->addDescriptor(new BLE2902());


pService->start();


// ----- FIX: Improve BLE stability -----

esp_ble_conn_update_params_t conn_params;


conn_params.latency = 0;

conn_params.min_int = 24; // 30 ms

conn_params.max_int = 40; // 50 ms

conn_params.timeout = 1000; // 10 sec supervision timeout

// NOTE: No min_ce_len / max_ce_len on some ESP32 SDKs

```

```
// Apply connection parameters
esp_ble_gap_update_conn_params(&conn_params);
// -----

pServer->getAdvertising()->start();
Serial.println("BLE started, advertising");
notifyTypeDetail("ready", nullptr);

}

void loop() {
    runSchedulerTick();
    delay(10);
}
```